Docker & Container Security

Oct 28, 2026 · CA-2

Assessment at a glance

You have 90 minutes to containerise a three-tier web app (frontend, backend and database), check it for vulnerabilities, and publish a signed, versioned backend image to Docker Hub. It is one continuous scenario: each stage builds on the one before, so work in order.

Item	Detail
Total marks	40
Duration	90 minutes
Mode	Individual, hands-on, on your own machine
AI tools	Allowed (see rules)
Submission	A public GitHub repository with your code and screenshots
Plagiarism	Zero marks for the whole assessment

Stage	What you do	Marks	Suggested time
1. Containerise the backend	Dockerfile, build, run	8	15 min
2. Compose the stack	Frontend, backend and database, two networks, a volume	10	25 min
3. Scan and fix	Trivy scan, fix the image, rescan	8	18 min
4. Version and publish	Version tags, Docker Hub, run from the registry	7	12 min
5. Sign and verify	cosign sign, verify, reject an unsigned release	7	12 min
Wrap-up	README, final commit, push, submit the URL	—	8 min

Stage	What you do	Marks	Suggested time
Total		**40**	**90 min**

Plagiarism policy: zero marks

Plagiarism of any kind earns 0 out of 40 for the whole assessment. The same penalty applies to every student involved, whether they copied or shared.

AI tools are allowed, so similar-looking code on its own is not treated as plagiarism. Copying is judged on evidence that only you can produce:

- Screenshots taken on another machine, reused from someone else, or edited (for example, the registration number in the prompt changed or cropped out).
- Another student's registration number, port, Docker Hub account, image digest, Git commit or cosign key anywhere in your submission.
- Sharing your files, screenshots, repository, Docker Hub image or keys with another student, or copying from theirs.

Rules and personalisation

AI tools are allowed. Use any assistant (ChatGPT, Claude, Copilot, Gemini or others) to write code, explain errors and find commands. You must run every command yourself, on your own machine. List the AI tools you used in your README; declared use costs no marks.

Personalise everything. Work out your values before you start and use them exactly as shown in every stage.

Value	Rule	Example for reg. no. 12345637
REGNO	Your registration number	`12345637`
PORT	80 + last two digits of REGNO	`8037`
Image	`<dockerhub-user>/fest-backend`	`rahul42/fest-backend`

Show your REGNO in every terminal. Set the prompt once in each new terminal window:

```
# bash
export PS1='[12345637] \w $ '
# zsh (macOS default)
export PS1='[12345637] %~ %# '
```

```
# powershell
function prompt { "[12345637] PS $(Get-Location)> " }
```

Screenshot rules

- Capture the whole window. Terminal screenshots must show your REGNO prompt and the command; browser screenshots must show the address bar and your Docker Hub username.
- Name each file with the evidence code given in the task, for example `S2-4.png`. One screenshot may cover two tasks if all the evidence is visible: name it with both codes, for example `S5-2_S5-3.png`.
- Keep them in a screenshots folder in your repository. Do not edit them, crop out the prompt or reuse anyone else's.

Windows tip: in PowerShell, type `curl.exe` rather than `curl`, which can run a different command, or use `Invoke-RestMethod`.

Commit after every stage; push only at the end. Finish each stage with a commit such as `Stage 3 done`, and push once, in the last 10 minutes. Your history still shows your progress stage by stage, and nobody can copy work in progress from your public repository.

Before the clock starts

Complete these checks before the 90 minutes begin; they are not timed and carry no marks. Tell the invigilator at once if any of them fails.

1. **Docker is running and you are signed in to Docker Hub.** `docker version` shows a Server section and `docker login` succeeds. Anonymous pulls are capped at 100 per 6 hours per IP address, and the whole lab shares one (Docker Hub pull limits).
2. **Trivy is installed** from the official installation page, and its database is downloaded with `trivy image --download-db-only`. Never use Trivy 0.69.4, 0.69.5 or 0.69.6: attackers published those builds with stolen credentials in March 2026 (Docker's incident note).
3. **cosign is installed:** `cosign version` works.
4. **Your GitHub repository is ready:** create a public repository named `<REGNO>-docker-assessment`, clone it, check that you can push to it, and work inside that folder.

5. **Base images are pulled:** `python:3.9`, `postgres:17-alpine` and `nginx:1.30-alpine`.

The scenario

You are the DevOps volunteer for FestPass, the registration app for your college's tech fest. By the end of this assessment it must run in containers, with a scanned, signed backend image on Docker Hub.

FestPass has three parts: a web page (frontend), a small Python API (backend) and a PostgreSQL database. Today it runs only on the developers' laptops, and registrations open next week.

Arjun, the fest's tech lead, puts it simply:

> "Containerise it, run all three parts together, and make sure we ship nothing with known critical vulnerabilities. Publish the backend to Docker Hub with a proper version and sign it, so we know exactly what we deploy."

Each stage builds on the previous one. Every task is marked on its own evidence, so if a step defeats you, note it in your README and carry on.

Starter code

The fest's student developers hand you four files. Save them in your cloned repository with exactly these paths, and change them only where a task tells you to.

backend/app.py

```
"""FestPass registration API (starter code)."""
import os
import signal
import socket
import sys
import threading
from contextlib import closing

import psycopg2
from flask import Flask, jsonify, request

app = Flask(__name__)

REGNO = os.getenv("REGNO", "NOT-SET")
APP_VERSION = os.getenv("APP_VERSION", "dev")
```

```
TABLE_SQL = """CREATE TABLE IF NOT EXISTS registrations (
    id SERIAL PRIMARY KEY,
    name TEXT NOT NULL,
    event TEXT NOT NULL,
    created TIMESTAMPTZ NOT NULL DEFAULT now())"""
_table_lock = threading.Lock()
_table_ready = False


def connect():
    return psycopg2.connect(
        host=os.getenv("DB_HOST", "localhost"),
        dbname=os.getenv("DB_NAME", "festpass"),
        user=os.getenv("DB_USER", "festpass"),
        password=os.getenv("DB_PASSWORD", ""),
        connect_timeout=3,
    )


def run_sql(sql, params=(), fetch=False):
    """Create the table on first use, run one statement, commit and close."""
    global _table_ready
    with _table_lock:
        if not _table_ready:
            with closing(connect()) as conn, conn, conn.cursor() as cur:
                cur.execute(TABLE_SQL)
            _table_ready = True
    with closing(connect()) as conn, conn, conn.cursor() as cur:
        cur.execute(sql, params)
        return cur.fetchall() if fetch else None


@app.errorhandler(psycopg2.Error)
def database_error(err):
    return jsonify(error="database unavailable"), 503


@app.get("/api/health")
def health():
    try:
        run_sql("SELECT 1")
        database = True
    except psycopg2.Error:
        database = False
    return jsonify(service="festpass-backend", regno=REGNO,
version=APP_VERSION,
```

```
                    container=socket.gethostname(), database=database)


@app.get("/api/registrations")
def list_registrations():
    rows = run_sql("SELECT id, name, event, created FROM registrations ORDER
BY id", fetch=True)
    return jsonify([{"id": i, "name": n, "event": e, "created":
c.isoformat(timespec="seconds")}
                    for i, n, e, c in rows])


@app.post("/api/registrations")
def add_registration():
    data = request.get_json(silent=True) or {}
    name = str(data.get("name", "")).strip()
    event = str(data.get("event", "")).strip()
    if not name or not event:
        return jsonify(error="name and event are required"), 400
    new_id = run_sql("INSERT INTO registrations (name, event) VALUES (%s, %s)
RETURNING id",
                     (name, event), fetch=True)[0][0]
    return jsonify(id=new_id, name=name, event=event), 201


if __name__ == "__main__":
    # Exit promptly on `docker stop` instead of waiting to be killed.
    signal.signal(signal.SIGTERM, lambda *_: sys.exit(0))
    app.run(host="0.0.0.0", port=5000)
```

backend/requirements.txt (the versions on the developers' laptops)

```
Flask==2.2.2
Werkzeug==2.2.2
psycopg2-binary==2.9.10
```

frontend/index.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>FestPass 2026</title>
  <style>
    body { font-family: system-ui, sans-serif; max-width: 640px; margin: 2rem
auto; padding: 0 1rem; }
```

```
    #status { padding: 0.5rem; background: #eef2ff; }
  </style>
</head>
<body>
  <h1>FestPass 2026 registrations</h1>
  <p id="status">Checking the backend…</p>
  <form id="form">
    <input id="name" placeholder="Your name" required>
    <select id="event">
      <option>Hackathon</option>
      <option>Robo Race</option>
      <option>Tech Quiz</option>
    </select>
    <button>Register</button>
  </form>
  <ul id="list"></ul>
  <script>
    const api = (path, options) => fetch("/api/" + path, options).then(r =>
r.json());

    async function refresh() {
      const status = document.getElementById("status");
      try {
        const h = await api("health");
        status.textContent = `REGNO ${h.regno} | backend ${h.version} on
${h.container} | database ${h.database ? "connected" : "not connected"}`;
        const rows = await api("registrations");
        document.getElementById("list").replaceChildren(...
(Array.isArray(rows) ? rows : []).map(r => {
          const li = document.createElement("li");
          li.textContent = `#${r.id} ${r.name} (${r.event})`;
          return li;
        }));
      } catch (err) {
        status.textContent = "Backend not reachable";
      }
    }

    document.getElementById("form").addEventListener("submit", async (e) => {
      e.preventDefault();
      await api("registrations", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ name: document.getElementById("name").value,
event: document.getElementById("event").value }),
      });
```

```
      document.getElementById("name").value = "";
      refresh();
    });

    refresh();
  </script>
</body>
</html>
```

frontend/nginx.conf

```
server {
    listen 80;

    location / {
        root /usr/share/nginx/html;
        index index.html;
    }

    # Requests for /api/... are passed to the backend service on port 5000
    location /api/ {
        proxy_pass http://backend:5000;
    }
}
```

The backend listens on port 5000 and reads these environment variables:

Variable	Meaning	Default
`REGNO`	Your registration number, shown on the page	`NOT-SET`
`DB_HOST`	Database host	`localhost`
`DB_NAME`, `DB_USER`	Database name and user	`festpass`
`DB_PASSWORD`	Database password	empty
`APP_VERSION`	Version shown on the page	`dev`

Its endpoints are `GET /api/health`, `GET /api/registrations` and `POST /api/registrations` with a JSON body such as `{"name": "Asha", "event": "Hackathon"}`. The page at `/` calls them through nginx.

Stage 1 — Containerise the backend (8 marks)

The developers insist on Python 3.9, "because that's what runs on our laptops". Build it their way first; Stage 3 checks it for vulnerabilities.

- **1.1 (3 marks)** Write `backend/Dockerfile` starting `FROM python:3.9`. Install `requirements.txt` before copying `app.py`, so the install step is cached between builds, and start the app with `CMD`. Add a `backend/.dockerignore`. Evidence: both files.
- **1.2 (2 marks)** Build the image as `fest-backend:0.1` and list it with its size. Evidence: `S1-2`.
- **1.3 (3 marks)** Run it in the background as a container named `backend`, publishing port 5000 on your PORT and passing your `REGNO`. Show `docker ps`, the `/api/health` response with your REGNO (`"database": false` is expected for now) and the container's logs, then remove the container. Evidence: `S1-3`.

Stage 2 — Compose the three-tier stack (10 marks)

Now run all three parts together. Arjun's one rule: only the backend may talk to the database.

- **2.1 (4 marks)** Write `compose.yaml` with three services:
  - `db`: `postgres:17-alpine`, with user and database `festpass`, the password read from a `.env` file, and its data in a named volume.
  - `backend`: built from `backend/`, with `DB_HOST=db`, the same password and your `REGNO`. It starts only once `db` passes a `pg_isready` healthcheck.
  - `frontend`: `nginx:1.30-alpine`, published on your PORT and started after `backend`. Bind-mount `frontend/index.html` into `/usr/share/nginx/html/` and `frontend/nginx.conf` as `/etc/nginx/conf.d/default.conf`.

  Keep `.env` out of Git and commit a `.env.example` instead. Evidence: `compose.yaml`, `.env.example`.
- **2.2 (2 marks)** Put `frontend` and `backend` on one network, and `backend` and `db` on another, so the frontend cannot reach the database. Show `ping -c 1 db` failing from inside the `frontend` container. Evidence: `S2-2`.
- **2.3 (2 marks)** Open `http://localhost:<PORT>` and add two registrations; the page shows your REGNO. Evidence: `S2-3`.
- **2.4 (2 marks)** Run `docker compose down`, then `docker compose up -d`: both registrations must still be listed. Also show the volume with `docker volume ls`. Evidence: `S2-4`.

Stage 3 — Scan and fix with Trivy (8 marks)

Before registrations open, the faculty coordinator wants proof that the backend image has no known critical vulnerabilities.

Trivy, Aqua Security's open-source scanner, is required. Aqua's commercial platform adds AI-powered remediation guides (Trivy vs Aqua); use it too if your lab has access, and any AI assistant may help you read the results.

- **3.1 (2 marks)** Scan `fest-backend:0.1` and save the report as `reports/trivy-before.txt`. Evidence: `S3-1` showing the severity totals, and the report.
- **3.2 (3 marks)** Fix the image: switch `backend/Dockerfile` to a current slim or alpine Python base pinned to a version (never `latest`), run the app as a non-root user, and upgrade the libraries in `requirements.txt`. Build `fest-backend:0.2`. Evidence: both files.
- **3.3 (3 marks)** Scan `fest-backend:0.2` and save `reports/trivy-after.txt`. Show that the command below exits with code 0, meaning no CRITICAL vulnerability that has a fix (check with `echo $?`, or `$LASTEXITCODE` in PowerShell). Add a before-and-after table of CRITICAL and HIGH counts and image size to your README. Evidence: `S3-3`, the report, README.

```
trivy image --severity CRITICAL --ignore-unfixed --exit-code 1 fest-backend:0.2
```

Stage 4 — Version and publish to Docker Hub (7 marks)

The fixed image is ready to ship. Release it with proper version numbers, never `latest`.

- **4.1 (2 marks)** Rebuild the fixed image with `APP_VERSION=1.0.0`, tag it `<dockerhub-user>/fest-backend:1.0.0` and `<dockerhub-user>/fest-backend:1.0`, and push both tags to a public Docker Hub repository. Evidence: `S4-1`, showing the push output with the digest.
- **4.2 (2 marks)** Screenshot the repository's Tags page on Docker Hub, and record the `1.0.0` digest in your README. Evidence: `S4-2`, README.
- **4.3 (3 marks)** Change `compose.yaml` so `backend` uses `<dockerhub-user>/fest-backend:1.0.0` instead of building it. Delete your local backend images, run `docker compose up -d` so the image comes from Docker Hub, and reload the page: it must show backend `1.0.0` and your registrations. Evidence: `compose.yaml`, `S4-3`.

Stage 5 — Sign and verify with cosign (7 marks)

In March 2026, attackers used stolen credentials to push backdoored Trivy images to Docker Hub (Docker's incident note). From now on, FestPass runs only images that you have signed.

Use cosign. Docker Content Trust (`docker trust`) shuts down on 8 December 2026, so do not use it (Docker's retirement notice).

- **5.1 (1 mark)** Add `*.key` to `.gitignore`, then generate a password-protected cosign key pair and commit only `cosign.pub`. **If `cosign.key` appears in any commit, Stage 5 scores 0.** Evidence: `S5-1`, `cosign.pub`.
- **5.2 (2 marks)** Sign `fest-backend` version `1.0.0` by its digest, not its tag. Evidence: `S5-2`.
- **5.3 (2 marks)** Verify the signature with `cosign.pub`. Evidence: `S5-3`.
- **5.4 (2 marks)** Build and push `1.0.1` with `APP_VERSION=1.0.1`, without signing it, and show that `cosign verify` rejects it. Evidence: `S5-4`.

Submission

Push your final commit in the last 10 minutes, then submit your repository URL on the submission portal before the 90 minutes end. Marking uses only what is in your repository, so anything you have not pushed cannot earn marks. Lay it out like this:

```
<REGNO>-docker-assessment/
├── README.md
├── compose.yaml
├── .env.example
├── .gitignore
├── cosign.pub
├── backend/
│   ├── app.py
│   ├── requirements.txt
│   ├── Dockerfile
│   └── .dockerignore
├── frontend/
│   ├── index.html
│   └── nginx.conf
├── reports/
│   ├── trivy-before.txt
│   └── trivy-after.txt
└── screenshots/
    └── S1-2.png … S5-4.png
```

Your README must contain:

1. Your name, REGNO, PORT, Docker Hub username and Docker Hub repository link.
2. The AI tools you used and what you used them for.
3. The digest of `1.0.0` as pushed to Docker Hub.
4. The before-and-after vulnerability table from 3.3.
5. Any step you could not complete, one line each.

Before your final push:

- [ ] Every terminal screenshot shows your REGNO prompt, and every screenshot is in `screenshots/`.
- [ ] `cosign.key`, `.env`, Docker Hub passwords and access tokens appear in no commit.
- [ ] Your GitHub repository and your Docker Hub repository are both public.
- [ ] `1.0.0` is signed and `1.0.1` is not.

References

- Docker Hub pull usage and limits, Docker Docs
- Trivy supply chain compromise: what Docker Hub users should know, Docker, 23 March 2026
- Docker Content Trust: retirement and migration guidance, Docker, 16 June 2026
- Installing Trivy, Trivy docs
- Trivy OSS compared with Aqua, Trivy docs